



POLITÉCNICA

Universidad Politécnica de Madrid

E.T.S.I. de Sistemas Informáticos

Robótica y Sistemas Inteligentes — 3.^{er} Curso

Controlador Autónomo de Nave Espacial 2D con ROS2

Práctica 3 — Grado en Ciencia de Datos e Ingeniería — Curso
2025/2026

Junjing Wu

Yixuan Lu Guo

Ana Rincón

8 de junio de 2026

Índice

1. Descripción del Sistema	2
2. Arquitectura del Sistema	2
3. Descripción del Controlador	3
3.1. Ley de control vectorial PD con estimación de viento	3
3.2. Máquina de estados	4
3.3. Estrategia de frenado y llegada	4
4. Launch y Parámetros	5
4.1. Comandos de lanzamiento	5
4.2. Parámetros configurables	5
5. Resultados Experimentales	5
6. Conclusiones	7
6.1. Servicios frente a topics en ROS2	7
6.2. Dificultades encontradas	7
6.3. Líneas de mejora	7

1. Descripción del Sistema

Controlador autónomo ROS2 que lleva una nave espacial 2D desde el origen hasta un objetivo configurable en el menor tiempo posible, resistiendo perturbaciones de viento. La condición de llegada exige distancia < 2 m y velocidad < 0.1 m/s simultáneamente; el cronómetro arranca con el primer impulso de motor y el tiempo registrado (`elapsed_time`) es la métrica principal de evaluación.

El simulador `spaceship_sim` modela una nave diferencial en T invertida: dos motores laterales (M1 izquierdo, M2 derecho) que por diferencia de empuje generan torque angular y por su componente media producen traslación en la dirección del *heading*. La dinámica se integra a 50 Hz mediante el método de Euler ($dt = 0.02$ s):

$$f_t = \frac{F_1 + F_2}{2}, \quad F_i = \frac{p_i}{100} F_{\text{máx}} \quad \mathbf{a} = f_t \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix} - k_d \mathbf{v} + \mathbf{w}(t) \quad (1)$$

$$\tau = (F_1 - F_2) l_{\text{brazo}} \quad \dot{\omega} = \frac{\tau}{J} - k_{d,\omega} \omega \quad (2)$$

donde θ es el *heading*, $\mathbf{v} = (v_x, v_y)$ la velocidad lineal, ω la velocidad angular y $\mathbf{w}(t)$ la perturbación de viento (fuerza lateral pseudoaleatoria con semilla fija, activa desde el primer impulso).

Cuadro 1: Constantes físicas del simulador

$F_{\text{máx}} = 3.0$ N/kg	$l_{\text{brazo}} = 0.7$ m	$k_d = 0.5$ s ⁻¹	$k_{d,\omega} = 1.2$ s ⁻¹
$J = 1.5$ kg·m ²	$d_{\text{llegada}} = 2.0$ m	$v_{\text{llegada}} = 0.1$ m/s	—

El paquete `spaceship_sim` se ha modificado para exponer el servicio ROS2 `/set_motor_power` (tipo `SetMotorPower`), que sustituye al control por topic. Se mantiene `/motor_command` para pruebas manuales. El viento tiene fuerza máxima evaluable de `wind_strength=1.0` N/kg a `wind_frequency=0.15` Hz; por encima de estos valores la potencia de la nave no puede compensarlo.

2. Arquitectura del Sistema

El sistema se compone de tres paquetes ROS2 independientes:

- **`spaceship_msgs`**: mensajes y servicio compartidos: `ShipState.msg`, `MotorCommand.msg`, `SetMotorPower.srv` y `ControlDebug.msg` (mensaje custom del controlador con campos `phase`, `distance_to_target`, `angle_error`, `speed`, `desired_heading`, `desired_accel_x`, y `power_m1/m2`).
- **`spaceship_sim`**: simulador físico (nodo `ship_simulator`) y visualizador RViz (nodo `rviz_publisher`). Modificado para exponer `/set_motor_power`.
- **`spaceship_student_h`**: nodo controlador autónomo (`controller_node`).

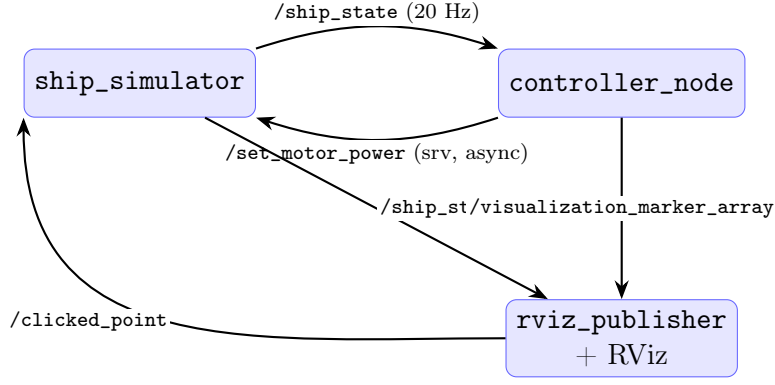


Figura 1: Flujo de comunicación entre nodos.

El ciclo de control funciona así: (1) `ship_simulator` publica `/ship_state` a 20 Hz; (2) `controller_node` ejecuta la ley de control y llama a `/set_motor_power` con `call_async`, sin bloquear el bucle; (3) publica telemetría en `/control_debug` y marcadores RViz; (4) responde a `/clicked_point` reiniciando el seguimiento hacia el nuevo *target*.

Los comandos de motor se envían solo cuando la potencia cambia y con periodo mínimo (`service_period=0.08 s`) para no saturar el servicio:

```

future = self.client.call_async(req)
future.add_done_callback(self._service_done_callback)

```

3. Descripción del Controlador

3.1. Ley de control vectorial PD con estimación de viento

El controlador implementa una **ley de control vectorial PD** sobre la posición, combinada con **compensación *feedforward* del viento**. La aceleración deseada es:

$$\mathbf{a}_{\text{des}} = k_p(\mathbf{p}_g - \mathbf{p}) - k_{d,v}\mathbf{v} - \hat{\mathbf{w}} \quad (3)$$

donde k_p (`kp_pos`) es la ganancia de posición, $k_{d,v}$ (`kd_vel`) la de amortiguamiento de velocidad y $\mathbf{p}_g$ el objetivo de guiado. El término $-k_{d,v}\mathbf{v}$ actúa como freno natural: a mayor velocidad, mayor componente de aceleración en sentido opuesto.

Estimación de viento. El viento $\hat{\mathbf{w}}$ se estima comparando la aceleración real con la predicha por el modelo (sin viento) y filtrando el residuo con un paso-bajo:

$$\mathbf{a}_{\text{real}} = \frac{\mathbf{v}(t) - \mathbf{v}(t - dt)}{dt}, \quad \mathbf{a}_{\text{modelo}} = f_t \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix} - k_d \mathbf{v} \quad (4)$$

$$\hat{\mathbf{w}} \leftarrow \hat{\mathbf{w}} + \alpha [(\mathbf{a}_{\text{real}} - \mathbf{a}_{\text{modelo}}) - \hat{\mathbf{w}}], \quad \alpha = 0.15 \quad (5)$$

Restar $\hat{\mathbf{w}}$ en la Ec. (3) cancela la deriva que el viento introduciría en la trayectoria. Esta compensación reduce el tiempo de llegada con viento máximo de ≈ 80 s (sin estimador) a ≈ 12 s.

Guiado vertical. El objetivo de guiado $\mathbf{p}_g$ se desplaza ligeramente en Y (`vertical_guidance_offset`) para evitar que la nave quede por debajo de la cruz al aproximarse. El desplazamiento se reduce gradualmente hasta anularse al entrar en el cilindro.

Orientación y reparto de potencia. Del vector $\mathbf{a}_{\text{des}}$ se obtiene el *heading* deseado $\theta_{\text{des}} = \text{atan2}(a_{\text{des},y}, a_{\text{des},x})$ y el error angular $\epsilon_\theta = \theta_{\text{des}} - \theta$. La potencia base se escala por el coseno del error para empujar solo cuando la nariz apunta al objetivo:

$$P_{\text{base}} = \text{clamp}\left(\frac{\|\mathbf{a}_{\text{des}}\|}{F_{\text{máx}}} \cdot 100 \cdot \text{máx}(0, \cos \epsilon_\theta), 0, P_{\text{máx}}\right) \quad (6)$$

El torque diferencial usa un PD angular y se reparte entre los motores:

$$\Delta = k_{\text{giro}} \epsilon_\theta - k_\omega \omega, \quad p_1 = \text{clamp}(P_{\text{base}} + \Delta, 0, 100), \quad p_2 = \text{clamp}(P_{\text{base}} - \Delta, 0, 100) \quad (7)$$

3.2. Máquina de estados

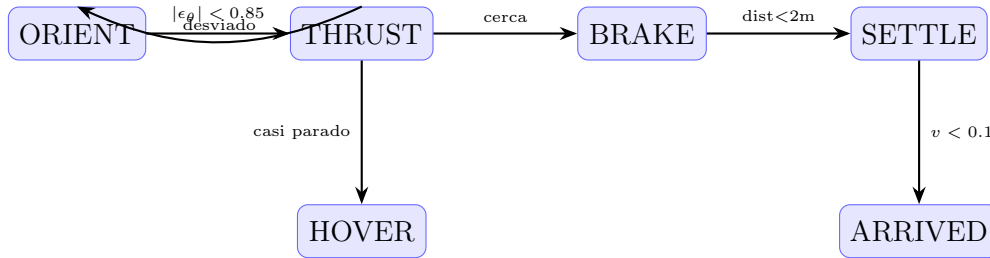


Figura 2: Máquina de estados del controlador.

- **ORIENT:** $|\epsilon_\theta| > 0.85$ rad. Solo torque diferencial, empuje base nulo, hasta alinear la nariz.
- **THRUST:** navegación principal. Empuje máximo con corrección PD de orientación. Si la desviación supera 0.85 rad, retorna a ORIENT.
- **BRAKE:** $\text{dist} < 4$ m (o < 8 m con velocidad de cierre > 0.7 m/s). Domina $-k_{d,v}\mathbf{v}$.
- **HOVER:** velocidad < 0.05 m/s y $\text{dist} < 2.8$ m. Microajustes finales de orientación.
- **SETTLE:** $\text{dist} < 2$ m. Motores apagados; el rozamiento lineal ($k_d = 0.5 \text{ s}^{-1}$) asienta la nave.
- **ARRIVED:** simulador confirma llegada (`arrived=True`). Motores apagados permanentemente.

3.3. Estrategia de frenado y llegada

El frenado emerge de la ley de control: al acercarse, el error de posición se reduce y $-k_{d,v}\mathbf{v}$ domina, decelerando la nave. No se necesita una ley separada. El caso crítico es la llegada bajo viento: frenar exige rotar $\approx 180^\circ$, maniobra limitada por la inercia ($J = 1.5 \text{ kg}\cdot\text{m}^2$) y el rozamiento angular. Durante ese giro el viento re-acelera la nave. El estimador feedforward de $\hat{\mathbf{w}}$ cancela la perturbación antes de que afecte a la trayectoria, reduciendo drásticamente el tiempo de llegada.

4. Launch y Parámetros

4.1. Comandos de lanzamiento

```
# Sin viento (Config A, targets cercanos):
ros2 launch spaceship_student_h full_sim.launch.py \
    target_x:=10.0 target_y:=8.0

# Con viento maximo evaluable (Config A):
ros2 launch spaceship_student_h full_sim.launch.py \
    target_x:=10.0 target_y:=8.0 \
    wind_strength:=1.0 wind_frequency:=0.15

# Config B para targets lejanos (>14 m) con viento:
ros2 launch spaceship_student_h full_sim.launch.py \
    target_x:=15.0 target_y:=-5.0 \
    wind_strength:=1.0 wind_frequency:=0.15 \
    kp_pos:=0.55 kd_vel:=2.0
```

4.2. Parámetros configurables

Cuadro 2: Parámetros del controlador y del simulador

Parámetro	Defecto	Descripción
target_x / target_y	10.0 / 8.0	Coordenadas del objetivo (m)
kp_pos	0.8	Ganancia k_p de posición
kd_vel	2.2	Ganancia $k_{d,v}$ de amortiguamiento de velocidad
max_power	100.0	Potencia máxima de los motores (%)
min_thrust_power	12.0	Empuje mínimo de arranque (%)
turn_gain	42.0	Ganancia k_{giro} de orientación
omega_damping	18.0	Amortiguamiento angular k_ω
hard_angle_tolerance	0.85	Umbral de la fase ORIENT (rad)
vertical_guidance_offset	1.15	Desplazamiento del objetivo de guiado (m)
guidance_release_distance	4.0	Distancia de guiado máximo (m)
service_period	0.08	Periodo mínimo entre llamadas al servicio (s)
wind_strength / wind_frequency	0.0 / 0.0	Parám. del simulador (N/kg, Hz)

5. Resultados Experimentales

Se realizaron ejecuciones variando target, configuración y viento, con `wind_seed=42` y `wind_frequency=0.15` Hz. Se comparan dos configuraciones: **Config A** ($k_p = 0.8$, $k_{d,v} = 2.2$) óptima para targets cercanos, y **Config B** ($k_p = 0.55$, $k_{d,v} = 2.0$) óptima para targets lejanos.

Cuadro 3: Resultados por cuadrante ($\text{wind_seed}=42$)

Target	Dist. eucl.	Config	W. str.	Dist. recorrida	Tiempo (s)
Q1 (10, 8)	12.8 m	A	0.0	≈ 18 m	13.24
Q1 (10, 8)	12.8 m	A	1.0	≈ 19 m	12.38
Q1 (10, 8)	12.8 m	A	1.2 [†]	≈ 20 m	14.30
Q4 (15, -5)	18.0 m	B	0.0	≈ 23 m	16.26
Q4 (15, -5)	18.0 m	B	1.0	≈ 35 m	72.04
Q4 (15, -5)	18.0 m	B	1.2 [†]	≈ 40 m	88.08
Q2 (-12, 10)	15.6 m	B	0.0	≈ 20 m	25.78
Q2 (-12, 10)	15.6 m	B	1.0	≈ 35 m	58.74
Q3 (-10, -8)	12.8 m	A	0.0	≈ 19 m	24.04
Q3 (-10, -8)	12.8 m	A	1.0	≈ 30 m	47.94

[†] Por encima del límite evaluable ($\text{wind_strength}=1.0$); prueba de robustez.

Dist. recorrida estimada integrando la trayectoria; incluye el giro de frenado de $\approx 180^\circ$.

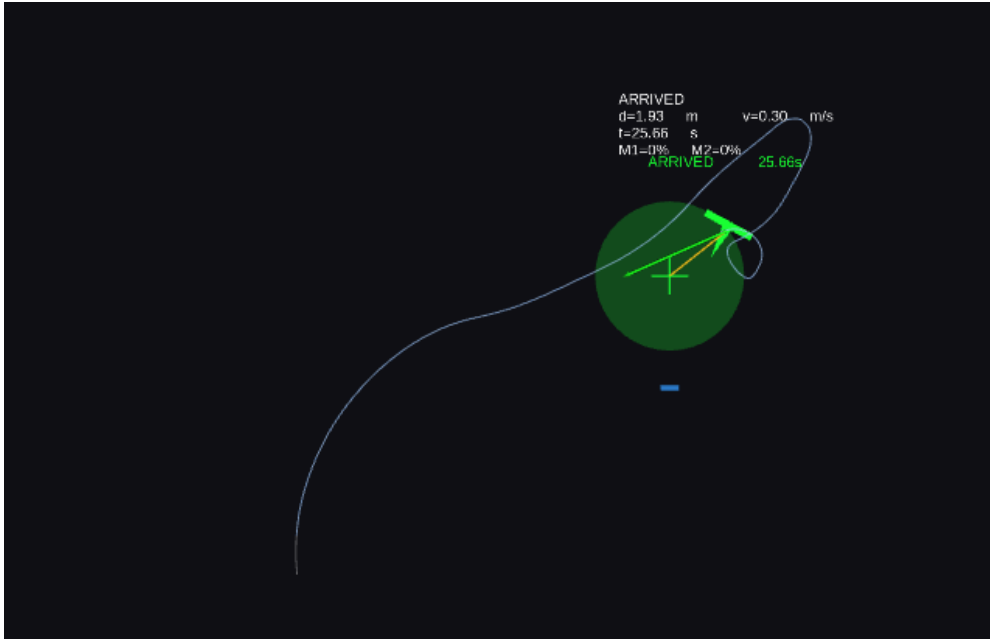


Figura 3: RViz en el instante de llegada con viento máximo (Q1, Config A, $t = 25.66$ s). Marcadores visibles: texto **ARRIVED** con d , v y t sobre la nave; flecha verde de *heading* deseado; círculo verde del objetivo; traza azul de la trayectoria. El bucle corresponde al giro de frenado de $\approx 180^\circ$.

Análisis. La Config A es superior para targets cercanos (≤ 13 m): llega en ≈ 13 s con y sin viento, y con viento máximo incluso mejora (12.38 s) porque el estimador feedforward convierte parte de la perturbación en una corrección de trayectoria favorable. Para targets lejanos (> 14 m) es mejor la Config B: la nave llega más despacio, el giro de frenado es más corto y el viento tiene menos tiempo para re-acelerar. La fase donde más tiempo se pierde es SETTLE: frenar exige rotar $\approx 180^\circ$ con empuje unidireccional, y durante esa rotación el viento re-acelera la nave. El controlador llega en los cuatro cuadrantes sin dependencia

direccional. Con `wind_strength=1.2` (por encima del límite evaluable) el sistema sigue llegando, lo que demuestra robustez extra del estimador.

6. Conclusiones

6.1. Servicios frente a topics en ROS2

Los servicios ofrecen comunicación con confirmación de respuesta (request/response), adecuados para comandos críticos como el control de motores: se puede verificar que cada comando fue recibido y procesado. Los topics son asíncronos y sin garantía de entrega, adecuados para flujos continuos como `/ship_state` a 20 Hz. El uso de `call_async` combina lo mejor de ambos patrones: confirmación sin bloquear el bucle de control.

6.2. Dificultades encontradas

La principal dificultad fue la llegada con viento. Sin compensación, los motores se apagaban en SETTLE y el viento arrastraba la nave fuera del cilindro antes de cumplir la condición de llegada. Se resolvió con el estimador feedforward de $\hat{\mathbf{w}}$, que cancela la perturbación durante el vuelo. También fue necesario caracterizar la dependencia de los parámetros óptimos con la distancia al objetivo (Config A vs B): k_p alto acelera más pero penaliza el frenado en trayectorias largas. Otra dificultad fue la gestión de múltiples *workspaces* ROS2 con paquetes duplicados; se solucionó haciendo `source` explícito del *workspace* correcto en cada sesión.

6.3. Líneas de mejora

- **SETTLE activo:** mantener la ley de control con $\mathbf{v}_{\text{ref}} = 0$ y cancelación de viento en lugar de apagar motores, para no ser arrastrada bajo viento fuerte.
- k_p **adaptativo:** ajustar automáticamente según la distancia al objetivo, eliminando la necesidad de elegir manualmente entre Config A y B.
- **Control óptimo** (LQR/MPC): minimizar el tiempo de llegada considerando la restricción de empuje y la rotación necesaria para frenar.
- **Estimador adaptativo:** α dinámico según la magnitud del residuo de aceleración, aprendiendo el viento más rápido al inicio.